

PRODUCT REQUIREMENTS DOCUMENT

Learning Management System with AI
Tutor
(LMS-AI)

Vertexon Learning Technologies Pvt. Ltd.

Field	Detail
Document Type	Product Requirements Document (PRD)
Document Version	v1.0
Status	Approved for Development
Project Duration	4 Weeks — Single Developer, Full Stack
Target Audience	Final-Year Students / Interns / Hackathon Teams / Portfolio Builders
Classification	Internal – Engineering Handoff

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. Problem Statement & Business Goals	4
2.1 Problem Statement	4
2.2 Business Goals	4
2.3 Success Metrics	4
3. Product Scope	5
3.1 In Scope (Build in 4 Weeks)	5
3.2 Explicitly Out of Scope for the 4-Week Build	5
4. User Personas	5
5. Functional Requirements	6
5.1 Student Module	6
5.2 AI Tutor Module	6
5.3 Instructor Module	6
5.4 Admin Module	6
6. Non-Functional Requirements	7
7. Database Schema Design	8
7.1 Entity List	8
7.2 Core Table Definitions (DDL-style)	8
7.3 Indexing Strategy	12
8. API Documentation	14
8.1 Auth	14
POST /auth/login — sample	14
8.2 Courses	14
8.3 Enrollment & Progress	14
8.4 Assignments & Quizzes	15
8.5 AI Tutor	15
POST /ai/chat/sessions/:id/messages — sample	15
8.6 Recommendations & Gamification	15
8.7 Admin	15
8.8 Standard Error Format	16
8.9 Rate Limits	16
9. System Architecture	17
9.1 High-Level Component Diagram	17
9.2 Request Flow — AI Tutor Question	17
9.3 Background Job Types (queue-based, Redis + worker)	17
9.4 Caching Strategy	18
10. Folder Structure	19
11. User Roles & Permissions Matrix	21

12. Development Roadmap — 4 Weeks, One Developer	22
Week 1 — Foundation, Auth, Core Data Layer.....	22
Week 2 — Learning Core: Lectures, Progress, Assignments, Quizzes.....	22
Week 3 — AI Tutor & Recommendations	22
Week 4 — Admin Panel, Discussions, Polish, Deployment.....	23
12.1 Risk Buffer	23
13. Suggested Technology Stack.....	24
14. Deployment Strategy	25
14.1 Environments	25
14.2 CI/CD Pipeline (GitHub Actions)	25
14.3 Infrastructure Components	25
14.4 Scaling Approach	25
14.5 Rollback Strategy.....	25
15. Future Enhancements.....	26
16. Tools Required.....	27
16.1 Development Environment	27
16.2 API & Testing Tools.....	27
16.3 Cloud & Infrastructure Accounts	27
16.4 AI/LLM Tooling.....	27
16.5 Design & Documentation	27
16.6 Monitoring & Ops.....	27

1. Executive Summary

Vertexon Learning Technologies is building an AI-augmented Learning Management System (LMS-AI) that goes beyond static video hosting and quiz delivery. The platform combines conventional LMS capabilities — course authoring, enrollment, assignments, assessments, certification — with an embedded AI tutor that uses Retrieval-Augmented Generation (RAG) over each course's own material to answer student questions, generate summaries, auto-create quizzes, and adapt difficulty to the learner.

The system is architected as a multi-tenant-ready SaaS product with three primary user classes — Students, Instructors, and Admins — each with a distinct workspace, and is built to be demoable end-to-end within four weeks by a single full-stack engineer, while still reflecting the architectural patterns (RBAC, background jobs, vector search, caching, containerization) expected of a production system.

This document is the single source of truth for scope, data model, APIs, architecture, roles, delivery plan, stack, deployment, and tooling for the build.

2. Problem Statement & Business Goals

2.1 Problem Statement

Traditional LMS platforms (Moodle-style, corporate e-learning portals) are passive content repositories. Students consume video and text but have no immediate, contextual help when they get stuck, no adaptive pacing, and no visibility into their own weak areas until a graded assessment tells them — often too late to act on it.

2.2 Business Goals

- Reduce the "doubt resolution" latency for a learner from hours/days (waiting on an instructor or forum reply) to seconds, using an AI tutor grounded in the actual course content.
- Increase course completion rate through personalized study plans, streaks, and gamification.
- Give instructors data-driven visibility into where students are struggling, at a topic level, not just a course level.
- Provide admins with a governance layer (course approval, content moderation, subscription/revenue tracking) suitable for a platform that intends to onboard third-party instructors.

2.3 Success Metrics

Framed as KPIs a real product would track once live:

Metric	Target
Course completion rate	+25% vs. baseline (no-AI control group)
Average doubt resolution time	Under 10 seconds (AI tutor response)
Quiz auto-generation accuracy (human-reviewed)	≥ 85% usable without edits
Student daily active usage (7-day streak retention)	≥ 40%
API p95 latency (non-AI endpoints)	Under 300ms

3. Product Scope

3.1 In Scope (Build in 4 Weeks)

- Auth (JWT-based, role-based access control)
- Student dashboard, course catalog, enrollment, progress tracking
- Video lecture streaming (pre-recorded, chunked/progressive delivery)
- Notes, bookmarks, assignment submission, quiz/assessment engine
- Certificates (auto-generated PDF), streaks, badges
- AI tutor chat (RAG-based, context-aware, per-course knowledge base)
- AI-generated lesson summaries, flashcards, auto-quiz generation
- Difficulty-adjusted explanations (beginner / intermediate / advanced)
- Personalized recommendation engine (rule-based + embedding similarity)
- Instructor course authoring, material upload, analytics dashboard
- Admin panel: user management, course approval, platform analytics, role management, content moderation
- Discussion forum (per-course, threaded)
- Notifications (in-app + email)
- Dark mode, basic i18n scaffolding

3.2 Explicitly Out of Scope for the 4-Week Build

Documented as Future Enhancements in Section 15:

- Live video conferencing / collaborative whiteboard
- Real-time peer-to-peer study rooms
- Full payment gateway / subscription billing integration
- Native mobile apps
- Full multilingual UI translation (only scaffolding included)
- Offline-first sync for downloaded content
- AI plagiarism detection engine (stubbed as an extension point)

This scoping split is intentional: it keeps the 4-week single-developer build realistic while preserving an architecture that can absorb the advanced features later without a rewrite.

4. User Personas

Ananya, Final-Year B.Tech Student. Enrolled in 4 online courses simultaneously. Needs quick doubt resolution at 11 PM when no TA is online, and wants to know exactly which topics she's weak in before her exam.

Rohit, Working Professional Instructor. Teaches a DSA course on the side. Needs to upload content once and get analytics on where students drop off, without manually grading every quiz.

Meera, Platform Admin. Manages instructor onboarding, approves new courses before they go live, and needs a single dashboard for platform health and revenue.

5. Functional Requirements

5.1 Student Module

ID	Requirement	Priority
FR-S1	User can register/login via email+password with JWT session issuance	P0
FR-S2	Student dashboard shows enrolled courses, % progress per course, streak counter	P0
FR-S3	Course catalog supports category, difficulty, and rating filters + search	P0
FR-S4	Video player supports resume-from-last-position and playback speed control	P0
FR-S5	Student can create timestamped notes and bookmarks on any lecture	P1
FR-S6	Student can upload assignment files (PDF/ZIP/code) before a deadline	P0
FR-S7	Quiz engine supports MCQ, multi-select, short-answer with auto-grading for objective types	P0
FR-S8	On 100% module completion, system auto-generates a downloadable certificate (PDF)	P1
FR-S9	Badges awarded on milestones (first course completed, 7-day streak, quiz perfect score)	P2
FR-S10	Recommendation widget suggests next course/topic based on quiz performance	P1

5.2 AI Tutor Module

ID	Requirement	Priority
FR-A1	Chat interface lets a student ask a question about the current course	P0
FR-A2	AI response grounded only in that course's ingested material via RAG, with source citation	P0
FR-A3	AI can summarize a selected lecture transcript into key points	P0
FR-A4	AI generates a personalized study plan from a student's quiz score history	P1
FR-A5	AI auto-generates a quiz (5–10 questions) from a lecture transcript for instructor review	P1
FR-A6	AI adjusts explanation depth: Beginner / Intermediate / Advanced	P1
FR-A7	AI generates flashcards (Q/A pairs) per module for spaced revision	P2
FR-A8	System tracks per-topic mastery score, feeding difficulty adjustment for chat and quizzes	P1

5.3 Instructor Module

ID	Requirement	Priority
FR-I1	Instructor can create a course with title, description, category, thumbnail, pricing tier	P0
FR-I2	Instructor can upload video/PDF/slide material per module/lecture	P0
FR-I3	Instructor can create assignments with rubric and deadline	P0
FR-I4	Instructor can build or approve AI-generated quizzes	P0
FR-I5	Instructor analytics dashboard shows per-lecture drop-off, avg. quiz score, time-on-task	P1
FR-I6	Instructor can post announcements visible to all enrolled students	P2

5.4 Admin Module

ID	Requirement	Priority
FR-AD1	Admin can approve/reject a newly submitted course before it is publicly listed	P0
FR-AD2	Admin can view/suspend/delete user accounts	P0
FR-AD3	Admin dashboard shows platform-wide metrics: DAU, enrollments, completion rate, revenue	P1
FR-AD4	Admin can assign/revoke roles (student/instructor/admin)	P0
FR-AD5	Admin can moderate reported forum posts/comments	P2

6. Non-Functional Requirements

Category	Requirement
Performance	Non-AI API endpoints respond within 300ms at p95 under 100 concurrent users
Scalability	Stateless backend services; horizontal scaling behind a load balancer
Security	Passwords hashed with bcrypt/argon2; short-lived JWT access + refresh tokens; RBAC middleware; input validation on all endpoints
Availability	Target 99.5% uptime for core LMS services (excludes best-effort AI endpoints)
Data Privacy	Student PII encrypted at rest; access logs retained for audit
Accessibility	WCAG 2.1 AA — keyboard navigation, ARIA labels, color-contrast compliance
Observability	Structured logging, request tracing, health-check endpoints per service
Caching	Redis caching for catalog listing, course metadata, and leaderboard reads
Rate Limiting	Per-user and per-IP rate limits on auth and AI-chat endpoints

7. Database Schema Design

Relational database (PostgreSQL) is used as the system of record; a vector database (pgvector extension) holds embeddings for RAG. Redis is used for caching and job queues, not as a system of record.

7.1 Entity List

users, roles, user_roles, courses, modules, lectures, enrollments, lecture_progress, notes, bookmarks, assignments, assignment_submissions, quizzes, quiz_questions, quiz_options, quiz_attempts, quiz_answers, certificates, badges, user_badges, streaks, ai_chat_sessions, ai_chat_messages, document_chunks (embeddings), study_plans, flashcards, recommendations, discussion_threads, discussion_posts, announcements, notifications, course_approvals, payments, audit_logs

7.2 Core Table Definitions (DDL-style)

```
-- USERS & AUTH -----
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  full_name VARCHAR(150) NOT NULL,
  email VARCHAR(180) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  avatar_url TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE roles (
  id SERIAL PRIMARY KEY,
  name VARCHAR(30) UNIQUE NOT NULL -- student, instructor, admin
);

CREATE TABLE user_roles (
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  role_id INT REFERENCES roles(id) ON DELETE CASCADE,
  PRIMARY KEY (user_id, role_id)
);

-- COURSES -----
CREATE TABLE courses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  instructor_id UUID REFERENCES users(id),
  title VARCHAR(200) NOT NULL,
  description TEXT,
  category VARCHAR(80),
  difficulty VARCHAR(20), -- beginner/intermediate/advanced
  thumbnail_url TEXT,
  price NUMERIC(10,2) DEFAULT 0,
  status VARCHAR(20) DEFAULT 'pending', -- pending/approved/rejected/archived
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE modules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id UUID REFERENCES courses(id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  order_index INT NOT NULL
);

CREATE TABLE lectures (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

    module_id UUID REFERENCES modules(id) ON DELETE CASCADE,
    title VARCHAR(200) NOT NULL,
    video_url TEXT,
    transcript TEXT,
    duration_seconds INT,
    order_index INT NOT NULL,
    resource_urls TEXT[] -- PDFs, slides
);

-- ENROLLMENT & PROGRESS -----
CREATE TABLE enrollments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    course_id UUID REFERENCES courses(id),
    enrolled_at TIMESTAMPTZ DEFAULT now(),
    progress_percent NUMERIC(5,2) DEFAULT 0,
    UNIQUE (user_id, course_id)
);

CREATE TABLE lecture_progress (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    enrollment_id UUID REFERENCES enrollments(id) ON DELETE CASCADE,
    lecture_id UUID REFERENCES lectures(id),
    watched_seconds INT DEFAULT 0,
    completed BOOLEAN DEFAULT FALSE,
    last_watched_at TIMESTAMPTZ
);

CREATE TABLE notes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    lecture_id UUID REFERENCES lectures(id),
    timestamp_seconds INT,
    content TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE bookmarks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    lecture_id UUID REFERENCES lectures(id),
    timestamp_seconds INT,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- ASSIGNMENTS -----
CREATE TABLE assignments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id UUID REFERENCES courses(id),
    title VARCHAR(200),
    instructions TEXT,
    rubric JSONB,
    due_date TIMESTAMPTZ
);

CREATE TABLE assignment_submissions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    assignment_id UUID REFERENCES assignments(id),
    user_id UUID REFERENCES users(id),
    file_url TEXT,
    submitted_at TIMESTAMPTZ DEFAULT now(),
    grade NUMERIC(5,2),
    feedback TEXT
);

-- QUIZZES -----

```

```

CREATE TABLE quizzes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  module_id UUID REFERENCES modules(id),
  title VARCHAR(200),
  is_ai_generated BOOLEAN DEFAULT FALSE,
  generated_from_lecture_id UUID REFERENCES lectures(id)
);

CREATE TABLE quiz_questions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quiz_id UUID REFERENCES quizzes(id) ON DELETE CASCADE,
  question_text TEXT,
  question_type VARCHAR(20), -- mcq/multi_select/short_answer
  order_index INT
);

CREATE TABLE quiz_options (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  question_id UUID REFERENCES quiz_questions(id) ON DELETE CASCADE,
  option_text TEXT,
  is_correct BOOLEAN DEFAULT FALSE
);

CREATE TABLE quiz_attempts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quiz_id UUID REFERENCES quizzes(id),
  user_id UUID REFERENCES users(id),
  score NUMERIC(5,2),
  started_at TIMESTAMPTZ DEFAULT now(),
  submitted_at TIMESTAMPTZ
);

CREATE TABLE quiz_answers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  attempt_id UUID REFERENCES quiz_attempts(id) ON DELETE CASCADE,
  question_id UUID REFERENCES quiz_questions(id),
  selected_option_ids UUID[],
  text_answer TEXT,
  is_correct BOOLEAN
);

-- GAMIFICATION -----
CREATE TABLE certificates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  course_id UUID REFERENCES courses(id),
  certificate_url TEXT,
  issued_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE badges (
  id SERIAL PRIMARY KEY,
  name VARCHAR(80),
  description TEXT,
  icon_url TEXT
);

CREATE TABLE user_badges (
  user_id UUID REFERENCES users(id),
  badge_id INT REFERENCES badges(id),
  earned_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_id, badge_id)
);

CREATE TABLE streaks (
  user_id UUID PRIMARY KEY REFERENCES users(id),

```

```

        current_streak INT DEFAULT 0,
        longest_streak INT DEFAULT 0,
        last_active_date DATE
    );

-- AI TUTOR -----
CREATE TABLE ai_chat_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    course_id UUID REFERENCES courses(id),
    mode VARCHAR(20) DEFAULT 'intermediate',
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE ai_chat_messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID REFERENCES ai_chat_sessions(id) ON DELETE CASCADE,
    sender VARCHAR(10), -- user/ai
    content TEXT,
    source_lecture_ids UUID[],
    created_at TIMESTAMPTZ DEFAULT now()
);

-- VECTOR STORE (pgvector) -----
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE document_chunks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id UUID REFERENCES courses(id),
    lecture_id UUID REFERENCES lectures(id),
    chunk_text TEXT,
    embedding VECTOR(1536),
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX ON document_chunks USING ivfflat (embedding vector_cosine_ops);

-- RECOMMENDATIONS & STUDY PLANS -----
CREATE TABLE study_plans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    course_id UUID REFERENCES courses(id),
    plan_json JSONB,
    generated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE flashcards (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    module_id UUID REFERENCES modules(id),
    question TEXT,
    answer TEXT
);

CREATE TABLE recommendations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    recommended_course_id UUID REFERENCES courses(id),
    reason TEXT,
    score NUMERIC(5,2),
    created_at TIMESTAMPTZ DEFAULT now()
);

-- DISCUSSIONS, ANNOUNCEMENTS, NOTIFICATIONS -----
CREATE TABLE discussion_threads (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id UUID REFERENCES courses(id),
    created_by UUID REFERENCES users(id),

```

```

        title VARCHAR(200),
        created_at TIMESTAMPTZ DEFAULT now()
    );

CREATE TABLE discussion_posts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    thread_id UUID REFERENCES discussion_threads(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id),
    content TEXT,
    is_flagged BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE announcements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id UUID REFERENCES courses(id),
    posted_by UUID REFERENCES users(id),
    content TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    title VARCHAR(150),
    body TEXT,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- ADMIN / GOVERNANCE -----
CREATE TABLE course_approvals (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id UUID REFERENCES courses(id),
    reviewed_by UUID REFERENCES users(id),
    decision VARCHAR(20), -- approved/rejected
    comment TEXT,
    reviewed_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    course_id UUID REFERENCES courses(id),
    amount NUMERIC(10,2),
    status VARCHAR(20), -- success/failed/refunded
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    actor_id UUID REFERENCES users(id),
    action VARCHAR(100),
    entity VARCHAR(50),
    entity_id UUID,
    metadata JSONB,
    created_at TIMESTAMPTZ DEFAULT now()
);

```

7.3 Indexing Strategy

- B-tree indexes on all foreign keys (course_id, user_id, module_id, lecture_id)
- Composite index on (user_id, course_id) in enrollments (already unique-constrained)

- ivfflat vector index on document_chunks.embedding for approximate nearest-neighbor search
- Partial index on courses.status = 'pending' for fast admin approval queue lookups
- GIN index on discussion_posts.content if full-text search is added later

8. API Documentation

Base URL: /api/v1. All endpoints except auth and public catalog require Authorization: Bearer <JWT>.

8.1 Auth

Method	Endpoint	Description
POST	/auth/register	Register a new user (default role: student)
POST	/auth/login	Returns access + refresh token
POST	/auth/refresh	Issues a new access token
POST	/auth/logout	Invalidates refresh token
GET	/auth/me	Returns current authenticated user profile

POST /auth/login — sample

```
// Request
{ "email": "ananya@example.com", "password": "SecurePass123" }

// Response 200
{
  "access_token": "eyJhbGciOi...",
  "refresh_token": "dGhpc2lzYXJl...",
  "user": { "id": "uuid", "full_name": "Ananya Sharma", "role": "student" }
}
```

8.2 Courses

Method	Endpoint	Description	Role
GET	/courses	List/filter/search catalog (category, difficulty, q, page)	Public
GET	/courses/:id	Course detail with modules/lectures	Public
POST	/courses	Create a course (status defaults to pending)	Instructor
PUT	/courses/:id	Update course metadata	Instructor (owner)
POST	/courses/:id/modules	Add module	Instructor (owner)
POST	/modules/:id/lectures	Add lecture (multipart upload)	Instructor (owner)
POST	/courses/:id/approve	Approve/reject course	Admin

8.3 Enrollment & Progress

Method	Endpoint	Description
POST	/courses/:id/enroll	Enroll current student
GET	/enrollments/me	List my enrolled courses with progress
POST	/lectures/:id/progress	Update watched-seconds / mark complete
POST	/lectures/:id/notes	Add a note
POST	/lectures/:id/bookmarks	Add a bookmark

8.4 Assignments & Quizzes

Method	Endpoint	Description
POST	/assignments	Create assignment (instructor)
POST	/assignments/:id/submit	Submit file (student)
PUT	/submissions/:id/grade	Grade a submission (instructor)
POST	/quizzes	Create quiz manually (instructor)
POST	/quizzes/:id/attempt	Start an attempt (student)
POST	/attempts/:id/submit	Submit answers, receive auto-graded score

8.5 AI Tutor

Method	Endpoint	Description
POST	/ai/chat/sessions	Start a chat session scoped to a course
POST	/ai/chat/sessions/:id/messages	Send a message, get RAG-grounded AI reply
POST	/ai/lectures/:id/summarize	Generate a lecture summary
POST	/ai/lectures/:id/generate-quiz	Auto-generate a quiz draft from transcript
POST	/ai/modules/:id/flashcards	Generate flashcards for a module
POST	/ai/study-plan	Generate a personalized study plan from quiz history
PUT	/ai/chat/sessions/:id/mode	Switch explanation mode (beginner/intermediate/advanced)

POST /ai/chat/sessions/:id/messages — sample

```
// Request
{ "message": "Why does quicksort degrade to O(n^2) on sorted input?" }

// Response 200
{
  "reply": "Because the pivot chosen (commonly the first or last element)...",
  "sources": [
    { "lecture_id": "uuid", "lecture_title": "Sorting Algorithms - Part 2", "timestamp_seconds":
842 }
  ],
  "mode": "intermediate"
}
```

8.6 Recommendations & Gamification

Method	Endpoint	Description
GET	/recommendations/me	Personalized course/topic suggestions
GET	/users/me/badges	Earned badges
GET	/users/me/streak	Current and longest streak
GET	/leaderboard/:courseId	Course leaderboard by quiz score/engagement

8.7 Admin

Method	Endpoint	Description
GET	/admin/users	List/search all users
PUT	/admin/users/:id/role	Assign/revoke role
PUT	/admin/users/:id/suspend	Suspend account
GET	/admin/courses/pending	Approval queue
GET	/admin/analytics/overview	Platform-wide metrics
GET	/admin/moderation/flagged-posts	Reported forum content

8.8 Standard Error Format

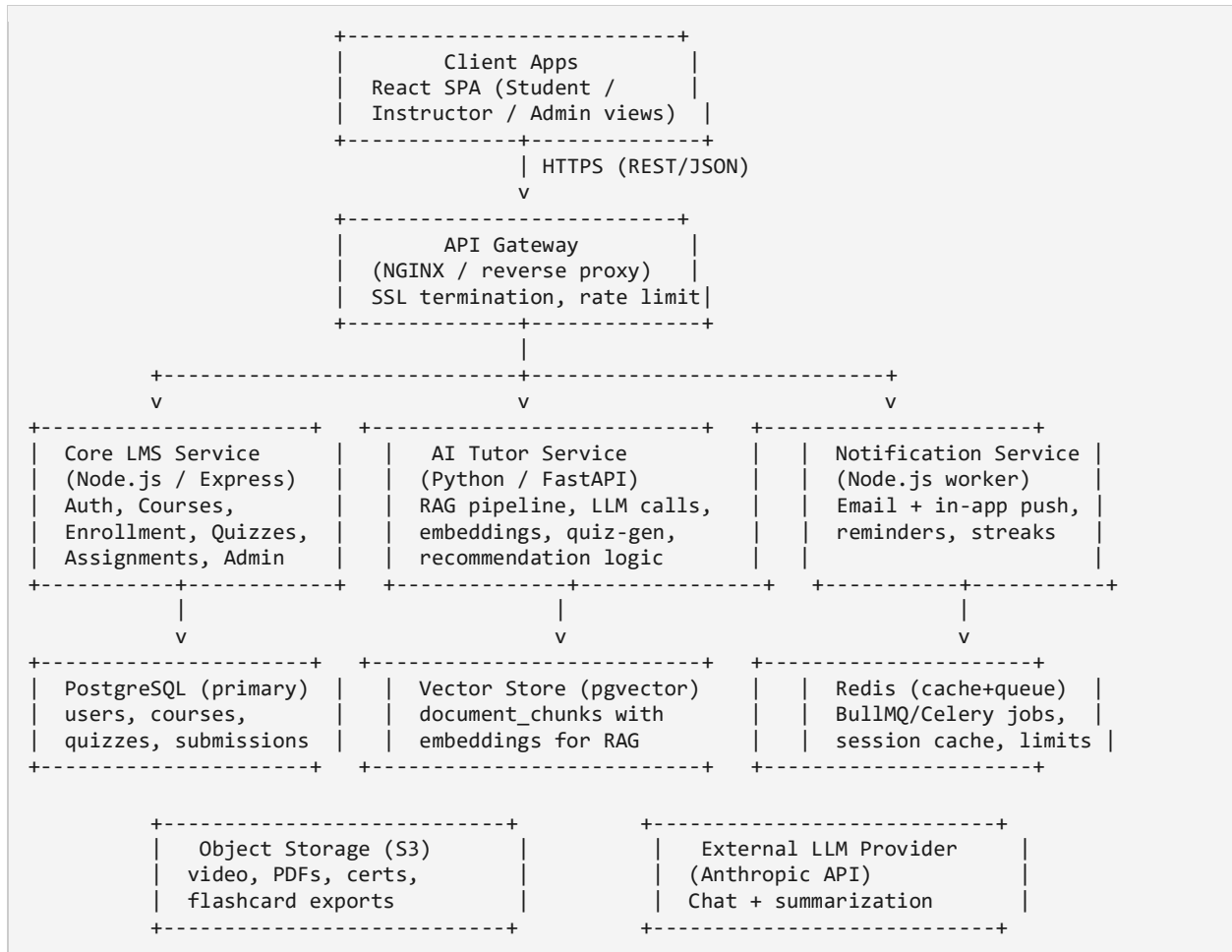
```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "email must be a valid email address",
    "field": "email"
  }
}
```

8.9 Rate Limits

Endpoint group	Limit
/auth/*	10 requests / minute / IP
/ai/chat/*	20 requests / minute / user
All other authenticated endpoints	100 requests / minute / user

9. System Architecture

9.1 High-Level Component Diagram



9.2 Request Flow — AI Tutor Question

1. Client sends the student's question to the AI Tutor Service with `session_id`.
2. Service embeds the question and performs a similarity search against `document_chunks` filtered by `course_id`.
3. Top-k retrieved chunks + question are assembled into a prompt and sent to the LLM provider.
4. Response, along with source lecture references, is stored in `ai_chat_messages` and returned to the client.
5. A background job updates the student's per-topic mastery score based on the interaction.

9.3 Background Job Types (queue-based, Redis + worker)

- Video transcription (speech-to-text) after upload
- Document chunking + embedding generation on new lecture material
- Nightly recommendation-score recalculation
- Streak evaluation (daily cron)

- Certificate PDF generation on course completion
- Email/notification dispatch

9.4 Caching Strategy

- Redis cache for course catalog listings (TTL 5 min, invalidated on course update/approval)
- Redis cache for leaderboard reads (TTL 60s)
- CDN in front of object storage for video/static assets

10. Folder Structure

```

lms-ai-platform/
|-- frontend/
|   |-- public/
|   |-- src/
|       |-- components/
|           |-- common/
|           |-- student/
|           |-- instructor/
|           |-- admin/
|           |-- ai-tutor/
|       |-- pages/
|           |-- auth/
|           |-- dashboard/
|           |-- course-catalog/
|           |-- course-player/
|           |-- quizzes/
|           |-- admin-panel/
|       |-- store/                # Redux Toolkit / Zustand slices
|       |-- hooks/
|       |-- services/            # API client wrappers (axios)
|       |-- layouts/
|       |-- utils/
|       |-- styles/
|       |-- routes/
|       |-- App.tsx
|-- package.json
|-- vite.config.ts

-- backend/
    |-- src/
        |-- modules/
            |-- auth/
            |-- users/
            |-- courses/
            |-- enrollments/
            |-- assignments/
            |-- quizzes/
            |-- gamification/
            |-- discussions/
            |-- notifications/
            |-- admin/
        |-- middleware/           # auth, rbac, rate-limit, error handler
        |-- jobs/                # BullMQ processors
        |-- config/
        |-- db/
            |-- migrations/
            |-- seeders/
        |-- utils/
        |-- server.ts
    |-- tests/
    |-- package.json
    |-- Dockerfile

-- ai-service/
    |-- app/
        |-- routers/
            |-- chat.py
            |-- summarize.py
            |-- quiz_gen.py
            |-- flashcards.py
            |-- study_plan.py
        |-- rag/
            |-- chunking.py
            |-- embeddings.py
  
```

```

|-- | | `-- retriever.py
|-- | | `-- recommendation/
|-- | | `-- models/
|-- | | `-- core/          # config, LLM client
|-- | | `-- main.py
|-- | requirements.txt
|-- | `-- Dockerfile
|-- infra/
|-- | `-- docker-compose.yml
|-- | `-- k8s/            # optional, deployment manifests
|-- | `-- nginx/
|-- | `-- terraform/     # optional IaC for cloud resources
|-- .github/
|-- | `-- workflows/
|-- | | `-- ci.yml
|-- | | `-- deploy.yml
|-- docs/
|-- | `-- prd.md
|-- | `-- api-spec.yaml   # OpenAPI 3.0
|-- | `-- architecture.md
|-- .env.example
|-- README.md

```

11. User Roles & Permissions Matrix

Capability	Student	Instructor	Admin
Register / Login	Yes	Yes	Yes
Browse course catalog	Yes	Yes	Yes
Enroll in a course	Yes	No	No
Watch lectures / track progress	Yes	No	No
Create notes/bookmarks	Yes	No	No
Submit assignments	Yes	No	No
Attempt quizzes	Yes	No	No
Use AI tutor chat	Yes	Yes (as reviewer)	No
Create/edit own course	No	Yes	Yes (any course)
Upload lecture material	No	Yes (own course)	Yes
Create/approve AI-generated quiz	No	Yes (own course)	Yes
Grade assignments	No	Yes (own course)	Yes
View own-course analytics	No	Yes	Yes
View platform-wide analytics	No	No	Yes
Approve/reject submitted courses	No	No	Yes
Manage user roles	No	No	Yes
Suspend/delete accounts	No	No	Yes
Moderate forum content	No	Yes (own course threads)	Yes (platform-wide)
Manage revenue/subscriptions	No	No	Yes

Enforcement: role checks happen at the API middleware layer (requireRole()), and ownership checks (e.g., "instructor owns this course") happen at the service layer before any mutation.

12. Development Roadmap — 4 Weeks, One Developer

The plan assumes ~6-7 focused hours/day. Each week ends with a working, demoable increment — this is deliberate so the project is never in a broken state for more than a day.

Week 1 — Foundation, Auth, Core Data Layer

Day	Tasks
1	Repo setup, monorepo/folder structure, Docker Compose (Postgres, Redis), CI skeleton
2	Database schema migration scripts for users/roles/courses/modules/lectures
3	Auth module: register, login, JWT issuance, refresh token, RBAC middleware
4	Course CRUD APIs + catalog listing/filtering, pagination
5	Frontend scaffold (routing, layout, auth pages, protected routes)
6-7	Student dashboard UI + course catalog UI wired to backend; buffer/testing

Milestone: *Users can register, log in, and browse/enroll in courses. Instructor can create a course.*

Week 2 — Learning Core: Lectures, Progress, Assignments, Quizzes

Day	Tasks
8	Video upload to object storage + streaming playback in course player
9	Lecture progress tracking, notes, bookmarks
10	Assignment creation + submission + manual grading flow
11	Quiz builder (manual) + attempt + auto-grading engine for objective questions
12	Certificates (PDF generation), badges, streak logic (daily cron job)
13-14	Instructor analytics dashboard (drop-off, avg. score); buffer/testing

Milestone: *A student can complete a full course loop: watch, note, submit assignment, take quiz, earn a certificate.*

Week 3 — AI Tutor & Recommendations

Day	Tasks
15	Set up AI service (FastAPI), integrate LLM provider client
16	Document ingestion pipeline: transcript chunking + embedding generation into pgvector
17	RAG retriever + chat endpoint with source citation
18	Lesson summarization + flashcard generation endpoints
19	Auto quiz generation from transcript + instructor review/approval UI
20-21	Difficulty-mode switching, mastery-score tracking, study plan + recommendation engine; buffer/testing

Milestone: *AI tutor answers course-grounded questions with citations; auto-generates quizzes, summaries, flashcards, and a study plan.*

Week 4 — Admin Panel, Discussions, Polish, Deployment

Day	Tasks
22	Admin panel: user management, role assignment, course approval queue
23	Admin analytics dashboard (platform-wide metrics), content moderation for forum
24	Discussion forum (threaded), announcements, in-app + email notifications
25	Dark mode, accessibility pass, i18n scaffolding, rate limiting, caching pass
26	Containerize all services, write CI/CD pipeline (GitHub Actions), staging deploy
27	Load testing (basic), bug fixes, security review (JWT expiry, input validation, RBAC edge cases)
28	Final polish, seed demo data, README + documentation, production deploy

Milestone: Full platform deployed to a cloud environment, all three role workspaces functional end-to-end, CI/CD pipeline green.

12.1 Risk Buffer

Days 6-7, 13-14, and 20-21 are intentionally kept as buffer/testing days within their respective weeks — if a task overruns, it absorbs there rather than cascading into the next week's scope.

13. Suggested Technology Stack

Layer	Technology	Notes
Frontend	React 18 + TypeScript, Vite	Fast dev server, strong typing
State Management	Redux Toolkit (or Zustand)	Predictable global state for auth/course/session data
Styling	Tailwind CSS	Utility-first, fast to theme (incl. dark mode)
Frontend Data Fetching	TanStack Query (React Query)	Caching, retries, background refetch
Backend (Core LMS)	Node.js + Express (or NestJS) + TypeScript	REST APIs, RBAC middleware
Backend (AI Service)	Python + FastAPI	Best ecosystem fit for RAG/embedding pipelines
Database	PostgreSQL 15+	Relational integrity for LMS entities
Vector Store	pgvector (Postgres extension)	Avoids running a separate vector DB at this scale
Cache / Queue	Redis + BullMQ (Node) / Celery (Python)	Caching + background jobs
Object Storage	AWS S3 (or MinIO for local dev)	Video, PDFs, certificates
LLM Provider	Anthropic API (Claude models)	Chat, summarization, quiz generation
Auth	JWT (access + refresh), bcrypt/argon2	Stateless auth
Video Delivery	HLS via a CDN in front of S3	Progressive/adaptive streaming
Speech-to-Text	Whisper (self-hosted) or managed STT API	Lecture transcription
PDF Generation	Puppeteer / pdf-lib	Certificates
Containerization	Docker + Docker Compose	Local dev parity with prod
Orchestration (optional)	Kubernetes (minikube / managed K8s)	Only if scaling beyond single-node deploy
CI/CD	GitHub Actions	Lint, test, build, deploy pipeline
Monitoring	Prometheus + Grafana (or hosted APM)	Metrics, health checks
Logging	Winston (Node) / structlog (Python)	Structured logs, centralized via Loki/CloudWatch
Testing	Jest, Supertest, Pytest, Playwright	Backend, AI service, frontend/E2E

14. Deployment Strategy

14.1 Environments

- Local: Docker Compose spins up Postgres, Redis, MinIO (S3-compatible), backend, ai-service, frontend.
- Staging: Single cloud VM or managed container service, mirrors production config with smaller resource limits.
- Production: Managed container hosting (managed Kubernetes or a PaaS such as Render/Railway/AWS ECS) fronted by a load balancer and CDN.

14.2 CI/CD Pipeline (GitHub Actions)

1. On PR: lint, unit tests, type-check, build (frontend, backend, ai-service)
2. On merge to main: build Docker images, push to container registry, deploy to staging, run smoke tests
3. On tagged release: promote staging image to production via blue-green or rolling deploy

14.3 Infrastructure Components

- Load balancer / reverse proxy (NGINX or a managed ALB) terminating SSL
- Managed PostgreSQL (with automated backups + point-in-time recovery)
- Managed Redis instance
- S3 (or equivalent) bucket with lifecycle policies for old video/cert assets
- CDN in front of static/video assets
- Secrets manager for API keys (LLM provider key, DB credentials, JWT secret)

14.4 Scaling Approach

- Core LMS and AI services are stateless — scale horizontally behind the load balancer as traffic grows
- Read-heavy endpoints (catalog, leaderboard) protected by Redis caching before hitting Postgres
- AI service isolated from core LMS service so a slow/expensive LLM call cannot degrade core CRUD latency
- Database read replica introduced once analytics/reporting queries start competing with transactional load

14.5 Rollback Strategy

Each deploy is tagged with the Git commit SHA; the previous known-good image remains in the registry so a rollback is a redeploy of the prior tag, not a rebuild.

15. Future Enhancements

Ordered roughly by expected value-to-effort ratio for a post-4-week phase:

- 1. Live classes with video conferencing (WebRTC-based, e.g., via a managed SFU) and an integrated collaborative whiteboard.
- 2. Real-time peer-to-peer study groups with matchmaking based on course/topic overlap.
- 3. AI plagiarism detection for code and text assignment submissions, using embedding similarity against a submission corpus.
- 4. Full payment gateway integration (subscriptions, one-time course purchases, instructor payouts).
- 5. Native mobile apps (React Native) reusing the existing REST API surface.
- 6. Offline-first content sync for downloaded lectures on mobile.
- 7. Full multilingual UI + AI tutor responses in regional languages.
- 8. Advanced analytics: cohort analysis, predictive at-risk-student flagging.
- 9. Marketplace features: instructor payout dashboards, coupon codes, affiliate tracking.
- 10. Adaptive learning paths that dynamically re-order modules per learner rather than only adjusting quiz difficulty.
- 11. Voice-based AI tutor interaction (speech-in, speech-out) for accessibility and mobile use.
- 12. SSO / SAML integration for institutional (university) deployments.

16. Tools Required

16.1 Development Environment

- Node.js (LTS, v20+) and npm/pnpm
- Python 3.11+ and pip/poetry
- Docker Desktop (or Docker Engine + Compose on Linux)
- PostgreSQL client (psql, DBeaver, or TablePlus) for schema inspection
- Redis CLI or RedisInsight for queue/cache inspection
- Git and a GitHub account (for version control and CI/CD)
- Visual Studio Code (recommended extensions: ESLint, Prettier, Docker, Python, Thunder Client/Postman)

16.2 API & Testing Tools

- Postman or Insomnia — API endpoint testing and collection sharing
- Jest, Supertest, Pytest, Playwright — automated testing
- k6 or Apache JMeter — basic load testing before production deploy

16.3 Cloud & Infrastructure Accounts

- A cloud provider account (AWS, GCP, or Azure) for object storage, managed Postgres/Redis, and container hosting
- A container registry (Docker Hub, GitHub Container Registry, or the cloud provider's own registry)
- A domain name and DNS management (for staging/production URLs)
- An SSL certificate provider (Let's Encrypt via Certbot, or the cloud provider's managed certificates)

16.4 AI/LLM Tooling

- Anthropic API account and API key for LLM chat/summarization/quiz-generation calls
- An embeddings model/provider for the RAG pipeline
- pgvector extension enabled on the PostgreSQL instance

16.5 Design & Documentation

- Figma (or equivalent) for UI wireframes/mockups before frontend build
- draw.io / Excalidraw for architecture and ER diagrams
- Notion, Confluence, or a docs/ folder in the repo for keeping this PRD and API spec versioned alongside the code

16.6 Monitoring & Ops

- Grafana + Prometheus (self-hosted) or a hosted APM (e.g., Datadog free tier, New Relic) for metrics
- A centralized logging tool (Loki, or the cloud provider's native logging service)

- Uptime monitoring (e.g., UptimeRobot) for the production health-check endpoint

End of Document — Vertexon Learning Technologies Pvt. Ltd. — LMS-AI PRD v1.0